DATA 266 Homework 4.5

DATA 266 — Homework 4.5

Sneha Singh
SID4 = 0670 | SEED = 670 | SLICE = 670 | HP_ID = 4 | CLS_A = 0 | CLS_B = 5

Notebook: `hw4.5.ipynb`. GPU: NVIDIA L4 on Colab Pro. Part C skipped (optional). One GPU.

Same standing parameters as HW1. HP_ID is reported only. There is no second network. Every number below is from the executed Colab notebook and from `RUN_LOG.txt` / `METRICS.md`.

Setup

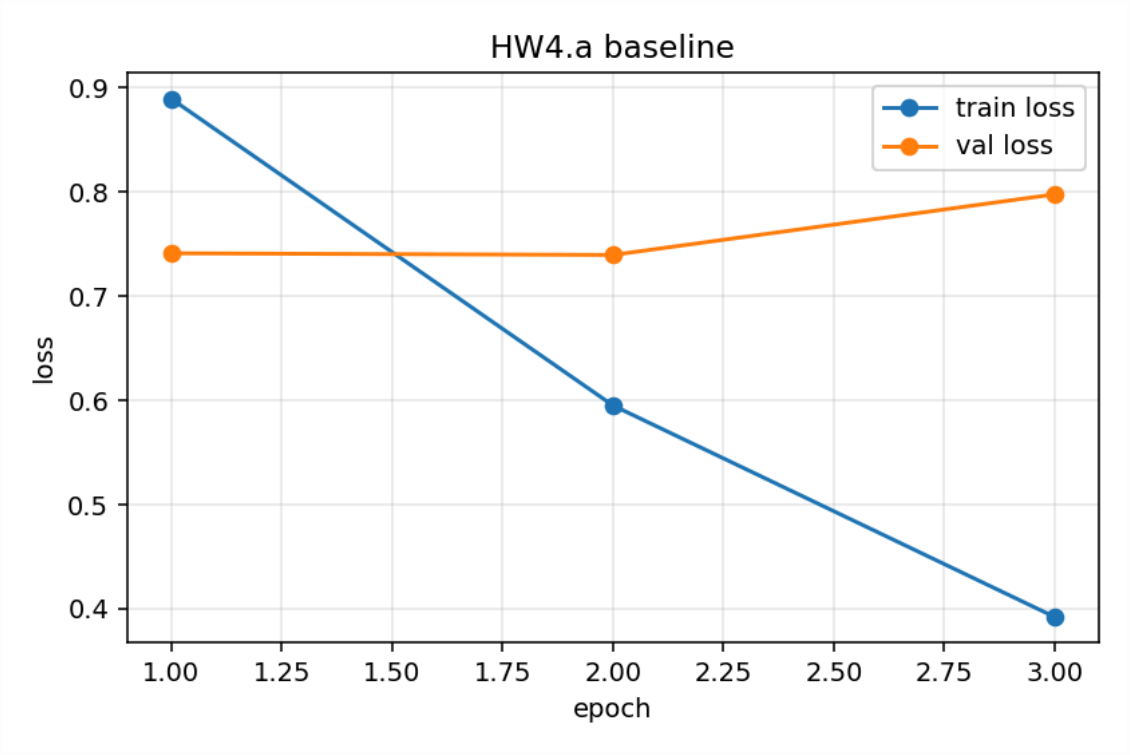
Item	Value
Data	`cardiffnlp/tweet_eval` sentiment, train split only
Rows	670–5669 inclusive (5,000 consecutive)
Split	3,500 / 750 / 750 with SEED 670
Labels in slice	negative 773, neutral 2290, positive 1937
Model	`bert-base-uncased`, BertTokenizer, max length 128
Baseline train	batch 16, AdamW, lr 2e-5, weight decay 0.01, 3 epochs
Serving model	`facebook/opt-125m`, 32 new tokens, greedy
Device	NVIDIA L4 (Colab Pro)

I do not use the dataset's own validation or test splits. The same frozen slice and the same SEED split are used for HW4.a and for every Part A / B run.

HW4.a — BERT sentiment baseline

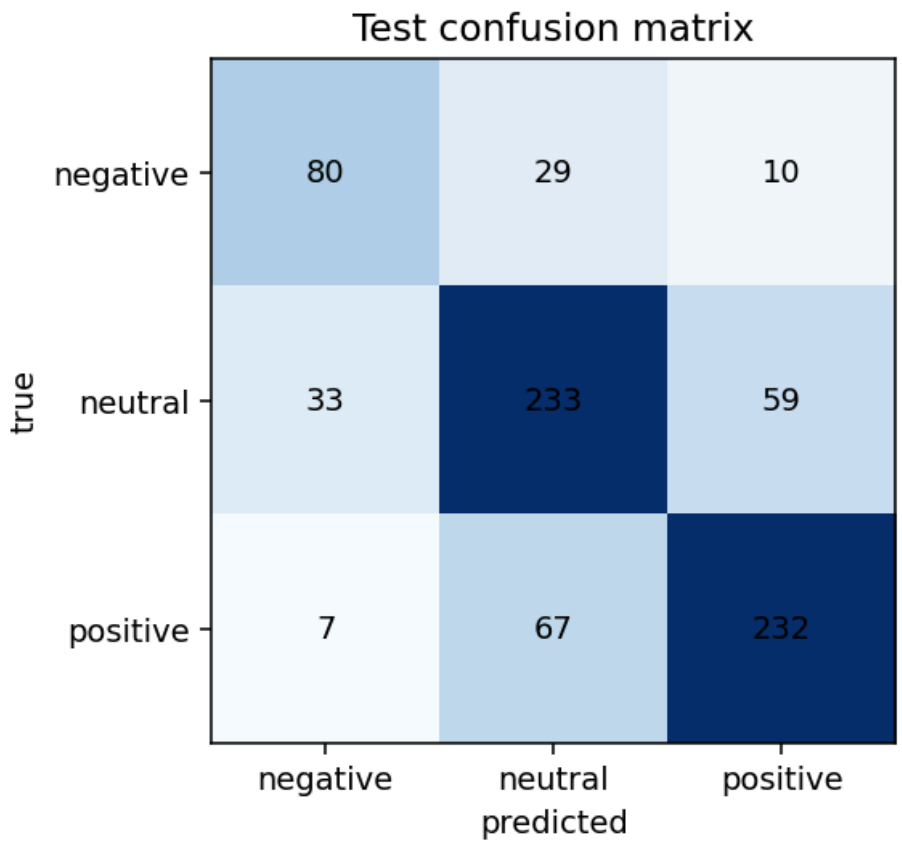
I fine-tuned BERT for three epochs. Validation loss was lowest at **epoch 2** (0.7395). Epoch 3 went to 0.7976, so I kept the epoch-2 weights for the test score. Training past that point cost about **38.3 s** (the third epoch).

Epoch	Train loss	Val loss	Val accuracy	Seconds
1	0.8892	0.7410	0.6640	37.0
2	0.5950	0.7395	0.6773	37.8
3	0.3922	0.7976	0.6813	38.3



Baseline loss

Peak allocated memory for the baseline was **2.48 GB**. Wall-clock training time was **113.1 s**. That is **0.0314 GPU-hours** (113.1 / 3600). Test accuracy at the best validation-loss epoch was **0.7267**.



Confusion matrix

Class	Precision	Recall	Support
negative	0.6667	0.6723	119
neutral	0.7082	0.7169	325
positive	0.7708	0.7582	306

Worst class: **negative**. That class has only 119 test examples (and 773 in the full 5,000-row slice), so the lower recall is partly a support problem, not only a model bug.

Five negative tweets the model called neutral:

1. *“Sophos is the Lamest Virus Ever…”* — insult is there, but the rest reads like a headline, so the model stayed neutral.
2. *“Because she’s Caitlyn Jenner. Come to FL sistah where you can run someone over with no consequences.”* — the negativity is implied and sarcastic; there is no clear negative word for the classifier to latch onto.
3. *“Tonight may be the night when I finally expose myself to the Gilmore Girls hysteria”* — ironic tone, not a clear complaint.
4. *“Monday was Gucci but then it went downhill”* — the tweet flips mid-sentence; the model missed the turn.

5. *“Hahaha 2nd place in Miss USA was Miss Texas…”* — sarcasm wrapped around a joke; it does not look like a plain negative tweet.

I saved one checkpoint only: `checkpoints/bert_baseline/`, after loading the best validation-loss state. Reload on three held-out sentences gave positive / negative / neutral in that order, which matches the labels I intended.

Serving — HF pipeline vs vLLM

Same 32 prompts, `facebook/opt-125m` (a different Hub model from BERT), float16, 32 new tokens. The first prompt is mine. Per-request latency for a batched vLLM call is the wait for that whole batch, because the requests return together.

Table HW4.1 — Serving comparison (HW4.a pass on the L4):

Configuration	Tokens/sec	p50 latency (ms)	p95 latency (ms)
HF pipeline, batch 1	115.7	269.3	280.8
vLLM, batch 1	97.8	319.1	328.4
vLLM, batch 8	627.8	349.2	354.9
vLLM, batch 32	2273.5	383.7	386.5

Part D re-run on the same L4 (HW4.5 pass):

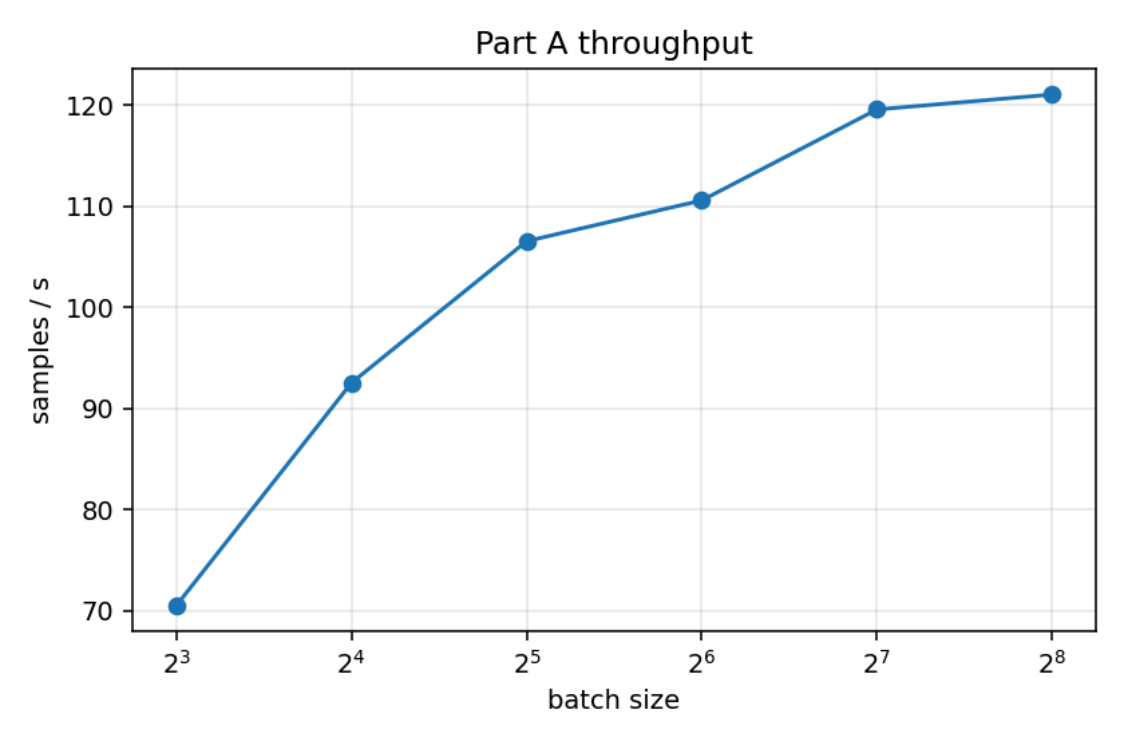
Configuration	Tokens/sec	p50 latency (ms)	p95 latency (ms)
vLLM, batch 1	97.9	320.4	325.3
vLLM, batch 8	622.9	348.8	355.4
vLLM, batch 32	2354.5	368.4	378.6

At batch 1, vLLM was already slower than the HF pipeline on both tokens/s and p50, so that row is not an improvement. At **batch 8**, p50 is still worse than the HF batch-1 baseline (349 ms vs 269 ms), but tokens/s jumps from 98 to 628, so total throughput is improving while per-request latency is worse. Batch 32 continues that pattern (higher tokens/s, higher p50). For an interactive application I care about **p50 latency**, because one user waits for their own reply. Tokens/s matters when many requests share the GPU and the product goal is aggregate capacity.

Part A — batch scaling

Same fine-tune settings. I doubled the batch until one forward/backward on exactly that many length-128 rows ran out of memory. Each size that fit also got one pass over the 3,500 training rows.

Batch	Samples/s	Peak GB	Seconds / epoch
8	70.5	2.09	49.7
16	92.5	2.49	37.8
32	106.5	3.76	32.9
64	110.6	6.15	31.7
128	119.6	11.00	29.3
256	121.0	20.65	28.9
512	OOM	—	—



Part A throughput

Largest batch that ran: **256**. Fastest batch: **256** at 121.0 samples/s. Throughput was already within 5% of that peak by batch **128** (119.6 samples/s), so the useful gains stop before the memory wall. Batch 512 is the first OOM.

I ran Part A on **one** card (Colab L4). The October 23 note says one GPU is enough, and Part C is optional, so there is no second-card throughput plot and no measured max-batch ratio. If I had two cards, that ratio would still not equal the VRAM ratio: model weights,

Adam state, and the CUDA context are a fixed cost on every card, activation memory is what scales with batch, and a power-of-two ladder skips sizes between the last success and the first OOM.

Part B — memory techniques

Table HW4.5.1. Max batch for the baseline row is Part A's max batch (256). Accumulation's "max batch" is the effective batch 128 with physical batch 8.

Technique	Max batch	Throughput (samp/s)	Peak memory (GB)	Eval accuracy
Baseline (HW4 settings)	256	92.8	2.48	0.7267
Mixed precision	256	189.3	2.26	0.7040
Gradient checkpointing	1024	70.7	2.09	0.7267
Gradient accumulation	128 (physical 8)	99.5	2.32	0.6520
8-bit optimiser	256	99.3	1.90	0.7187
Best combination	256	222.0	1.64	0.7160

Gradient checkpointing cut throughput from 92.8 to 70.7 samples/s (**–23.8%**) while saving only 0.39 GB at batch 16. That is the compute-for-memory trade the assignment asks for. It did raise the max batch to 1024.

Gradient accumulation used physical batch 8 × 16 steps = effective batch **128**. Accuracy was 0.6520. Wall-clock time to the **same number of optimiser steps** as the baseline was **816.9 s**, versus **113.1 s** for the baseline fine-tune. The lower peak (2.32 GB) is mostly the smaller physical batch, not a free memory trick.

Ranking by memory saved per unit of throughput lost, at the batch-16 training peaks (higher is better; "free" means memory fell and samples/s did not):

Rank	Technique	Memory saved (GB)	Throughput change (samp/s)	Saved GB / samp/s lost
1	8-bit optimiser	0.58	+6.5 (no loss)	free — apply first
2	Mixed precision	0.22	+96.5 (no loss)	free
3	Gradient accumulation	0.16	+6.7 (no loss)	free (smaller physical batch)
4	Gradient checkpointing	0.39	–22.1	0.018

I would apply **8-bit AdamW** first. On a **32 GB** card I would **not bother with gradient checkpointing** for this BERT run: it saves under 1 GB at batch 16 and is the only technique that clearly hurts throughput. The combination the ranking rule picked was **AMP + 8-bit AdamW** (no checkpointing): 222.0 samples/s, 1.64 GB, test accuracy 0.7160.

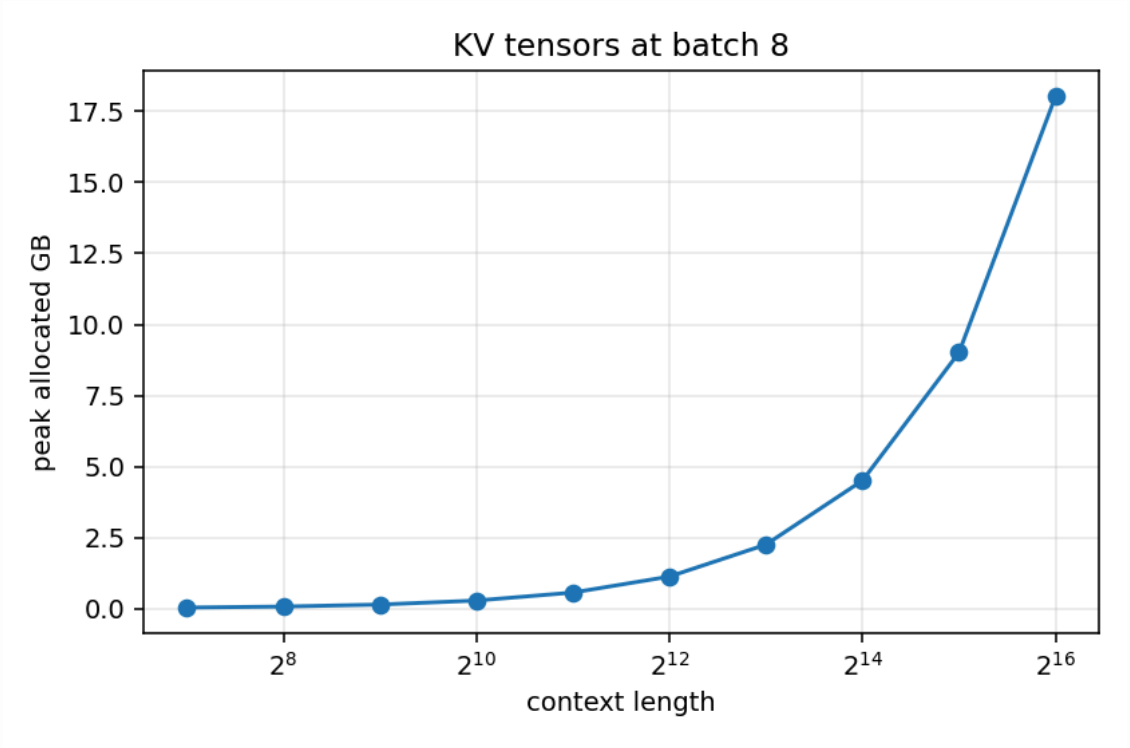
Part C

Part C skipped (optional).

Part D — serving revisited and KV cache

I re-ran the vLLM table on the same L4 (Table HW4.1 HW4.5 pass above). Compared with the HW4.a numbers from earlier in the same notebook, tokens/s and p50/p95 are nearly the same. That is expected: **hardware, precision (float16), model, and prompts did not change**. Any small differences are run-to-run noise, not a measurement flaw I need to correct.

I also allocated the key and value tensors for `facebook/opt-125m` at batch 8 across context lengths. That is the tensor size, not a vLLM serve.



KV cache

Context	Peak allocated GB
128	0.05

Context	Peak allocated GB
256	0.09
512	0.16
1024	0.30
2048	0.58
4096	1.14
8192	2.27
16384	4.52
32768	9.02
65536	18.02

Longest context at batch 8 that still fit: **65536** in this ladder. I did **not** hit an allocation OOM on the L4. OPT-125m's KV tensors are small (12 layers, 12 heads, head dim 64), so the tensor wall is past the model's usual 2048 position limit used in the vLLM serve (`max_model_len=2048`).

Part E — summary

GPU-hours for one full fine-tune on this L4:

[= = 0.0314]

That uses the HW4.a baseline wall-clock (batch 16, 3 epochs), not the Colab seat time.

The single configuration I would use for a full fine-tune on this hardware is **Best combination (AMP + 8-bit AdamW)**: peak **1.64 GB**, **222.0** samples/s, wall-clock **47.3 s**, test accuracy **0.7160** (within 1.5 points of the baseline 0.7267). It is the fastest row in Table HW4.5.1 that stays near baseline accuracy, and it does not need checkpointing on a card that already fits batch 256. This is the HW4.5 summary. It is not an HW5 baseline.

GPU identity for the deliverable: `results/GPU_CAPTURE.md` (two captures on the same L4 — training and serving). UUID was not printed by this notebook; the device name is NVIDIA L4.

Conclusion

On the L4, BERT-base at batch 16 is not memory-bound (2.48 GB). Throughput stops improving much by batch 128, and the card OOMs at 512. Mixed precision and 8-bit AdamW are the useful changes for this workload. Checkpointing is the wrong first tool

here unless I need a much larger batch. On the serving side, larger vLLM batches raise tokens/s while p50 gets worse than the HF batch-1 baseline, so an interactive demo should watch latency, not only throughput.